

MLOps Project – From basics to a mini Data Science Platform (Part 1)

Allister KOHN
allister.kohn@student-cs.fr

Enzo PALOS
enzo.palos@student-cs.fr

Elizaveta VASILEVA
elizaveta.vasileva@student-cs.fr

Jinsuh YOU
jinsuh.you@student-cs.fr

Introduction

The project proposed to us as part of our MLOps course aims to design an MLOps workflow to industrialize the lifecycle of Machine Learning models. A classic lifecycle includes data ingestion, model training, evaluation, and deployment. The goal is to identify and implement key features to facilitate the management, scalability, and reproducibility of models in production. The project is divided into two parts:

1. Write a specification detailing the chosen workflows and features
2. Implement the corresponding code, to be hosted on GitHub.

The objective is to structure a modular, clear, and reusable system, adopting a pragmatic and robust approach.

To this end, the GitHub repository [dsba-platform](#) is a relevant starting point, as it offers a modular architecture that integrates the essential steps of an MLOps workflow. It also includes a CLI, an API (with FastAPI), and a Dockerfile, facilitating containerization, integration, and deployment to the cloud. Furthermore, its structure easily allows for adding advanced features that ensure a smooth and reproducible workflow.

The code in this repository is suited to **binary classification** tasks, like the famous **Titanic** dataset, which predicts passenger survival based on their characteristics. Consequently, we have chosen to focus on a tool intended exclusively for binary classification tasks in order to remain focused on a clear and well-controlled scope.

Workflow features

Our goal is to create a **command-line tool** that allows Data Scientists to manage an MLOps workflow as simply as possible. We therefore want to implement various features:

- **Import and versioning of datasets:** Ability to add datasets from a local file or a URL, with management of successive versions to ensure traceability.
- **Data preprocessing:** Standardization of datasets with management of missing values, encoding, and normalization to ensure their quality before training.
- **Model training and saving:** Training of several binary classification models (XGBoost, Random Forest, Logistic Regression, SVM, Decision Tree), with recording of hyperparameters and performance.
- **Selection of the best model:** Automatic identification of the model offering the best performance according to a selected metric (Accuracy, Precision, Recall, F1-Score).
- **Prediction with trained models:** Generation of predictions from a test file using a specific model or the best available model.
- **Deployment and management of an API:** Creation of a REST API enabling interaction with the trained models and performing remote predictions.
- **Containerization:** Encapsulation of the API and models for easy and reproducible deployment.
- **Automated deployment to the cloud:** Hosting the API in a cloud environment, making the service accessible online with simplified management.

Finally, we will not forget to write detailed documentation so that Data Scientists can adopt this tool without excessive complexity and effectively.

Tools used

To implement our MLOps workflow, we will use several tools and technologies adapted to each stage:

- **Import and versioning of datasets:** AWS S3 for remote storage.
- **Data preprocessing:** Pandas and Scikit-Learn for data manipulation and cleaning, supporting encoding and handling missing values.
- **Model training and saving:** Scikit-Learn and XGBoost for model training, with an option for hyperparameter optimization via GridSearchCV.
- **Selection of the best model:** Comparison of models with Scikit-Learn, using performance metrics such as Accuracy, Precision, Recall, and F1-Score.
- **Prediction with trained models:** Loading models and running predictions via Scikit-Learn and joblib for model persistence.
- **Deployment and management of an API:** FastAPI to create a REST API for exposing models and making real-time predictions.
- **Containerization:** Docker to encapsulate the API and models, ensuring a reproducible and portable deployment.
- **Automated deployment:** AWS App Runner for hosting the API without server management, with Docker images stored on AWS ECR.

Conclusion

This document lays the foundation for our MLOps project by clearly defining the essential features and the tools to be used. It provides a structured vision of the workflow, from importing and managing datasets to deploying models in production. By adopting a modular approach and relying on proven technologies, we ensure the scalability and reproducibility of the system.